

It's 2am. A CVE drops.

Where is it running?

Open Component Model — open source, NeoNephos Foundation.



THE PAIN

Your release isn't a thing. It's a scavenger hunt.

- The image is in GHCR. The chart is in another registry.
- The SBOM is on an S3 bucket someone set up two years ago.
- The Terraform that wired it together is in a fourth repo.
- The CVE doesn't care.

Thirty minutes to find the pieces.

THE FRAGMENTATION

The release has no name.

**Every artifact has an identity.
None of them name the release.**

Mirror the image — the reference changes.

Mirror the chart — the reference changes.

Cosign signs each piece. None of them sign the release as one named, location-independent unit.

PACK Pack.

components:

- name: github.com/acme.org/helloworld
version: 1.0.0
provider:
name: acme.org

resources:

- name: image
type: ociImage
access:
type: OCIImage/v1
imageReference: ghcr.io/stefanprodan/podinfo:6.9.1

\$ ocm add cv

Produces a CTF archive.

Portable. OCI-compatible.

Move it with ocm transfer.

SIGN

What gets signed and travels.

component:

```
name: github.com/acme.org/helloworld
version: 1.0.0
provider:
```

```
  name: acme.org
```

resources:

```
- name: image
  type: ociImage
```

```
  digest:
```

```
    hashAlgorithm: SHA-256
```

```
    value: 70a2577d7b...
```

```
  access:
```

```
    type: OCIImage/v1
```

signatures:

```
- name: default
```

```
  digest: { value: 70a2577d7b... }
```

\$ ocm sign cv

OCM signs the descriptor,
not the individual files.

The descriptor carries digests
for every artifact — one signature,
tamper-evident over the release.

TRANSPORT Travels.

- Registry → Registry. Public to private, cloud to cloud.
- Registry → CTF archive. The whole release in one filesystem.
- CTF → Registry. Behind the air gap. No callback upstream.

Same identity in OCI, S3, or on USB. Verify with the same key.

DEPLOY

Apply once. The controllers take over.

Component CR

OCM Controllers

pull descriptor	<i>Repository</i>
verify signature	<i>Component</i>
resolve resources	<i>Resource</i>
apply & reconcile	<i>Deployer</i>

Cluster

COMPOSE

A product is a tree.

`github.com/acme/sovereign/product : 1.0.0`

```
|— github.com/acme/sovereign/notes      : 1.0.0
└— github.com/acme/sovereign/postgres   : 1.0.0
```

Leaf components carry resources. The product references them.

One name, one signature, covers the whole tree.

DAY 2

Bump the product. Everything follows.

spec:

```
version: 1.1.0    # was: 1.0.0
```

1. kro re-renders the RGD with the new spec.version.
2. Component CR's semver updates; controllers resolve and verify the new version.
3. Resource CRs resolve new digests for the child charts and images.
4. Flux HelmReleases roll the new artifacts.

You changed one line. The cluster did the rest.

ODG

The scanner speaks libraries. OCM speaks components.

WHAT THE SCANNER REPORTS

CVE-2026-XXXX in libfoo

[eu.gcr.io/acme/notes](#)
[@sha256:70a2577d...](#)

package · version · digest

WHAT ODG DISCOVERS

```
$ oras discover \  
  eu.gcr.io/acme/notes@sha256:70a2577d... \  
  --artifact-type \  
    application/vnd.ocm.software.ownership.v1+json
```

→ annotations:

```
software.ocm.component.name  
  = github.com/acme/notes  
software.ocm.component.version  
  = 1.0.0  
software.ocm.artifact  
  = { name: notes-image, kind: resource }
```

The image is unchanged. Ownership rides in a side-car.

AT 2AM

It's 2am. You already know.

SCANNER

CVE-2026-XXXX in libfoo
found in image [eu.gcr.io/acme/notes@sha256:70a2577d...](#)



ODG

oras discover the image with the OCM ownership artifactType
→ component github.com/acme/notes : 1.0.0 (resource notes-image)



DASHBOARD

owner = team-notes · env = eu-prod-12 · triaged = no



ACTION

Page team-notes. Patch on the shelf. Bump product. Done.

Thirty seconds. Coffee's still warm.

A release is a thing, not a scavenger hunt.

Star and watch — <https://github.com/open-component-model>

Try the tutorial — ocm.software/docs/getting-started

Talk to us — community channels on the website

ADOPT

Two paths. Pick the one that fits Monday.

FROM ZERO — 10 MINUTES

Install the ocm CLI.

Write a 10-line component-creator.yaml.

ocm add cv → sign → verify → transfer → verify.

Signed at source, verified at destination, transferable anywhere.

ON YOUR PLATFORM — WIRE IN

Install OCM controllers; point at your registry.

Apply a Component CR — verified, reconciling.

Compose with Argo, Flux as you already do.

Your next release ships as an OCM component.